JavaScript-02 第10回_まとめ

第10回の目標

- **実践的なJavaScriptのコードを書いて、Webサイトを動かそう！**
- 今回やるアニメーション
  - ハンバーガーメニュー
  - スライドイン
  - オープニングアニメーション

ハンバーガーメニューを動かす

仕様

- ハンバーガーメニューの3本線がバツ印（×）になる
- ナビゲーションが出現する
- ハンバーガーメニューが開いているときは、画面全体のスクロールが固定される（応用）
- もう一度ハンバーガーメニューをクリックすると元に戻る

すべて**クラスの付け外し**で実装する（第9回のDOM操作の3ステップ）。 1. HTMLから要素を取得する 2. クリックしたときにイベントを発火させる 3. クラスを付け外しする

JavaScript

```
const hamburger = document.querySelector(".js_hamburger");
const navigation = document.querySelector(".js_navigation");
const body = document.querySelector(".js_body");

// ハンバーガーをクリックしたら
hamburger.addEventListener("click", () => {
  // それぞれに対してis-activeクラスを付け外しする
  hamburger.classList.toggle("is-active");
  navigation.classList.toggle("is-active");
  body.classList.toggle("is-active");
});
```

- `function () {}` は `() => {}`（**アロー関数**）と書くこともできる。意味はほぼ同じで、短く書ける

- 1回のクリックで3つの要素のクラスを同時に付け外しし、CSS側で「変化前」と「`.is-active`がついた変化後」を用意しておくことで見た目が変わる

CSS（変化後の見た目）

```
/* 開いている間はページ全体のスクロールを固定する */
.js_body.is-active {
  overflow: hidden;
}

/* 3本線を×にする */
.js_hamburger-bar {
  transition: top 0.24s, transform 0.24s, opacity 0.24s;
}

.js_hamburger.is-active .m_hamburger-bar:first-child {
  top: 50%;
  transform: translateY(-50%) rotate(135deg);
}

.js_hamburger.is-active .m_hamburger-bar:nth-child(2) {
  opacity: 0; /* 真ん中の線は消す */
}

.js_hamburger.is-active .m_hamburger-bar:nth-child(3) {
  top: 50%;
  transform: translateY(-50%) rotate(-135deg);
}

/* ナビゲーションは普段は透明＋クリック不可にしておく */
.js_navigation {
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.24s;
}

.js_navigation.is-active {
  opacity: 1;
```

```
  pointer-events: auto;
}
```

- 上下の線を中央（`top: 50%`）に集めてから逆向きに回転させると×になる（第5回のハンバーガーメニューの作り）
- `pointer-events: none;`：要素をクリックできないようにする。`opacity: 0`で見えなくしただけだと、透明なナビゲーションがクリックを邪魔してしまうため一緒に指定する
- `body { overflow: hidden; }`でスクロールが固定されるが、iOSでは効かない場合があるので、実務では こちらの方法が推奨されている（「ハンバーガー 背景固定 ios」で調べる）
- 2本目の線は`transform: translate(50%, -50%);`で右に動かしながら`opacity: 0;`で消すと、スッと消える動きになる
- `.is-active`は直前にスペースを空けずに書く（`.js_nav.is-active`）

なぜis-activeがつくと動くのか

- `.js_navigation.is-active`は`.js_navigation`より詳細度が高いので、クラスがつくと変化前の記述を上書きする
- `transition: opacity 0.5s;`は「`opacity`だけを0.5秒かけて変化させる」書き方。`transition: 0.5s;`だと他のプロパティもすべて0.5秒かけて変化する

うまく動かないときの確認

状態	疑う場所	確認すること
検証ツールで`is-active`がつかない	JS	`.`と`,`、`:`と`;`の間違い、全角文字が入っていないか。コンソールのエラー
`is-active`はつくのに動かない	CSS	`.is-active`の前にスペースがないか、クラス名・プロパティ名、`;`忘れ
どこにもエラーがない	—	スーパーリロード（Mac：`command + Shift + R`／Win：`control + Shift + R`）。`console.log("ok!")`を途中に書いて、どこまで動いているか確かめる

- `is-active`がついているかは、検証ツールで確認できる（HTMLファイル自体は変わらない）
- GSAPを使っていて動かないときは、GSAPとJSが読み込まれているかをコンソールで確認する（ JSが読み込まれているか確認する方法）。困っている箇所より前の記述でエラーが出ていることもある

ハンバーガーメニューのスニペット

html.jsonに登録すると、`ham`と打つだけで呼び出せる。

```
"hamburger-menu": {
  "prefix": "ham",
  "body": [
    "<button class=\"m_hamburger js_hamburger\">",
    "\t<span class=\"m_hamburger-bar\"></span>",
    "\t<span class=\"m_hamburger-bar\"></span>",
    "\t<span class=\"m_hamburger-bar\"></span>",
    "</button>"
  ],
  "description": "hamburger-menu"
}
```

- 📎 ハンバーガーメニュー実装サンプル集／おしゃれなハンバーガーメニュー9選／開閉アニメーション4選／アニメサンプル集（卒業後）

取得した要素を変数に入れるメリット

- 管理がしやすい
- パフォーマンスが上がる（毎回HTMLから探し直さなくてよい）
- `js_`のクラスを使うことで、CSSの都合でクラス名が変わってもJSに影響が出ないようにできる
- どのセレクタがJSで使われているかわかりやすくなる
- `id`ではなくクラスで取得しているのは、CSS設計（詳細度を上げない、第4回参照）との兼ね合いのため

【発展】メニューをクリックしたらページ内のセクションへ移動し、メニューを閉じる

- 同じページ内のセクションに移動するには、移動先の`section`に`id`をつけ（`id="service"`）、ナビゲーションのリンクを`href="#service"`にする
- メニューを閉じるには、すべてのリンクに対してクリックイベントを設定し、`is-active`を外す

```
const navLinks = document.querySelectorAll(".l_header-nav_link");


navLinks.forEach((navLink) => {
  navLink.addEventListener("click", () => {
    hamburger.classList.remove("is-active");
    navigation.classList.remove("is-active");
    body.classList.remove("is-active");
  });
});
```

- `querySelectorAll`で複数取得した要素は、`forEach`で1つずつ取り出してイベントを設定する（第9回の繰り返し処理）
- 閉じるだけなので`toggle`ではなく`remove`を使う

📎 JavaScriptを使ったハンバーガーメニューを作ろう！

GSAPの導入

GSAP：JavaScriptでアニメーションを簡単に書けるようにするライブラリ。headタグ内で、自分の`main.js`より先に読み込む。

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.11.3/gsap.min.js"></script>
<script src="js/main.js" defer></script>
```

📎 GSAPの導入方法

- 読み込みのコードは公式サイトから最新のものを取得する（📎 GSAP公式 Installationの「Install Helper」で「CDN」を選ぶ）
- 【豆知識】GSAPは、LCPの改善に貢献するアニメーションライブラリとしてGoogleに推奨されている
    - LCP：ページの読み込み時に、画面内で一番大きなコンテンツが表示されるまでの時間を表す指標
    - 大きな画像の読み込みに時間がかかるとLCPは遅くなる。ユーザーが待たずにページを見られるよう改善していく大切な指標

Tween（1つのアニメーション）

メソッド	動き	使う場面
`gsap.to()`	今の状態 → 指定した状態（未来）へ動かす	消す
`gsap.from()`	指定した状態（過去） → 今の状態へ動かす	表示させる
`gsap.fromTo()`	指定した状態（過去） → 指定した状態（未来）へ動かす	両方指定して挙動を安定させたいとき

- 基本は`from`か`to`でよい（`fromTo`は使う機会が少ない）
- 違いがピンとこなければ、`from`と`to`を入れ替えて動きを比べてみる

```
// from：指定した状態（下・透明）から今の状態へ → フェードイン
gsap.from(".gsap_item-from", {
  y: 60,
```

```
  autoAlpha: 0,
  duration: 2,
});

// to：今の状態から指定した状態（下・透明）へ → フェードアウト
gsap.to(".gsap_item-to", {
  y: 60,
  autoAlpha: 0,
  duration: 2,
});
```

- 「今」＝HTML・CSSまで書き終わった、アニメーションがつく前の状態
  - 例：白い文字を出現させたい → 今すでに出ている状態に最終的に持っていきたいので`from`
  - 例：黒い背景を消したい → 今出ている状態から最終的に変えたいので`to`

よく使うプロパティ

プロパティ	意味
`duration`	アニメーションにかける秒数
`autoAlpha`	透明度。`opacity`と`visibility`をまとめて操作する（0で完全に非表示）
`x` / `y`	X軸／Y軸方向の位置（px）
`delay`	開始までの待ち時間（秒）
`onComplete`	アニメーションが完了したときに実行する関数

Tweenを扱う

autoAlphaとopacityの違い

- 違いは`visibility`がつくかどうか。`autoAlpha: 0`にすると`visibility: hidden;`もつく
- `autoAlpha`はGSAPでしか使えない

指定	状態
`opacity: 0`	透明だが、クリックできる
`autoAlpha: 0`	透明になり、クリックもできなくなる

`visibility: hidden;`の性質 - 透明になり見えなくなる - クリックできなくなる - 領域は残る（`display: none;`とは違う）

プロパティ	見える？	存在する？	触れる？
`opacity: 0;`	見えない	いる	触れる
`visibility: hidden;`	見えない	いる	触れない
`display: none;`	見えない	いない	触れない

- GSAPでは基本`autoAlpha`を使うと覚えてよい。`opacity`は透明でも要素が残るので、意図せずクリックされたり、他の要素に被さったりする
- 📎 opacity・display・visibilityの違い／display: none・opacity: 0・visibility: hiddenの違い
- 📎 GSAPの基本プロパティまとめ（透明度の変化）

スライドイン（Timeline）

仕様

- リロードすると、キャッチコピーとサブコピーが左 → 右にふわっと入ってくる
- 時間は0.6秒、コピー → サブコピーの順に時間差をつける

```
gsap
  .timeline()
  .from(".js_copy", {
    duration: 0.6, // 0.6秒かけて
    autoAlpha: 0,  // 透明度0の状態から
    x: -100,       // x軸方向に-100pxの位置から
  })
  .from(".js_sub-copy", {
    duration: 0.6,
    autoAlpha: 0,
    x: -100,
  });
```

- `gsap.timeline()`に`.from()`をつなげると、前のアニメーションが終わってから次が始まる → 時間差を簡単につけられる
- 📎 Timelineを扱う

timelineとdelayの比較

方法	特徴
`delay`（遅延時間を指定）	1つ目のアニメーションの時間が変わると、後ろのdelayもすべて計算し直す必要がある。アニメーションが多いほど面倒
`timeline`	1つ終わったら自動で次を始めるので、delayの計算がいらない → **timelineの方がよい**

- 【＋α】timelineの中でも、発火タイミングをずらして時差をつけられる（ GSAP timelineの発火タイミングの管理）

【発展】GSAPを使わずに同じ動きをつける

```
const mainCopy = document.querySelector(".js_copy");
const subCopy = document.querySelector(".js_sub-copy");

// ページの読み込みが完了したら
window.addEventListener("load", () => {
  mainCopy.classList.add("is-active");
  subCopy.classList.add("is-active");
});
```

```
.js_copy,
.js_sub-copy {
  opacity: 0;
  transform: translateX(-100px);
}

.js_copy.is-active,
.js_sub-copy.is-active {
  opacity: 1;
  transform: translateX(0);
  transition: 0.6s;
}

.js_sub-copy.is-active {
  transition-delay: 0.6s; /* 時間差 */
}
```

- `window`の`load`イベントで「ページの読み込みが完了したとき」に処理を実行できる

オープニングアニメーション

仕様

- 読み込んだときに、オープニングのテキストがふわっと入ってくる
- オープニングアニメーションが完了したら、オープニング画面が消える
- 消えた後、キャッチコピーとサブコピーが順番に左 → 右にふわっと入ってくる
- ＋α：アニメーション中はスクロールできないようにする
- 応用：一度ローディングしたら、それ以降はローディングしない

```
const opening = document.querySelector(".js_opening");


function OpeningAnime() {
  // アニメーション中はスクロールを固定（ハンバーガーで作ったoverflow: hiddenを再利用）
  body.classList.toggle("is-active");


  gsap
    .timeline()
    .from(".js_opening-txt", {
      duration: 1.2,
      autoAlpha: 0,
      y: 40, // 40px下から
    })
    .to(".js_opening", {
      autoAlpha: 0,
      duration: 0.6,
      delay: 1.2,
      // アニメーションが完了したらスクロール固定を解除
      onComplete: function () {
        body.classList.toggle("is-active");
      },
    })
    // スライドインのアニメーション
    .from(".js_copy", {
      duration: 0.6,
      autoAlpha: 0,
      x: -100,
    })
```

```
    .from(".js_sub-copy", {
      duration: 0.6,
      autoAlpha: 0,
      x: -100,
    });
}

OpeningAnime(); // 関数呼び出し
```

- オープニングのテキストは`.from()`（下からふわっと出す）、オープニング画面は`.to()`（今の状態から透明にして消す）
- タイムラインを使うことで、複数のアニメーションの順番をまとめて管理できる

GSAPのスニペット

GSAPは予測変換で出てこないことが多いので、javascript.jsonに登録しておくと便利。

```
{
  "gsapTo": {
    "prefix": "gsap-to",
    "body": "gsap.to(\".クラス名\"$1, {\"プロパティ:〇〇\"$2});",
    "description": "gsapのtoメソッド"
  },
  "gsapFrom": {
    "prefix": "gsap-from",
    "body": "gsap.from(\".クラス名\"$1, {\"プロパティ:〇〇\"$2});",
    "description": "gsapのfromメソッド"
  },
  "gsapTimeLine": {
    "prefix": "gsap-timeline",
    "body": "gsap.timeline().from(\".クラス名\"$1, {\"プロパティ:〇〇\"$2}).from(, {});",
    "description": "gsapのtimelineメソッド"
  },
  "autoAlpha": {
    "prefix": "autoAlpha",
    "body": "autoAlpha",
    "description": "gsapの透明度とvisibilityを操作"
  }
}
```

LCPとFCP

- LCPとFCPは、Web制作で必ず押さえておきたい表示速度の指標（ FCPとは？）
- GSAPを導入しているサイト集（卒業後）

アニメーションのつけ方に迷ったら

- ディズニーのアニメーションの12の原則を参考にする
- GSAPアニメーションギャラリー：GSAPの動きのサンプルを見られる
- GSAPはいろいろ触って遊んでみるとよい
- GSAP入門（後編）タイムライン制御やスクロール演出／GSAPで複雑なオープニングアニメーション

【応用】2回目以降はオープニングを表示しない（sessionStorage）

```
function webStorage() {
  if (sessionStorage.getItem("access")) {
    // 2回目以降のアクセス：アニメーションは実行せず、オープニングを消しておく
    opening.classList.add("is-active");
    body.classList.add("is-active");
  } else {
    // 初回アクセス：目印を保存してアニメーションを実行
    sessionStorage.setItem("access", 0);
    OpeningAnime();
  }
}

webStorage();
```

- `sessionStorage`：ブラウザのタブを閉じるまでの間、データを保存しておける仕組み
  - `setItem(キー, 値)`で保存、`getItem(キー)`で取り出す
- 「保存した目印があるか」で初回かどうかを判定し、処理を出し分けている
- sessionStorage（MDN）

本日のまとめ

1. JavaScriptのアニメーションは「要素を取得 → イベントを指定 → クラスを付け外し」が基本（ハンバーガーメニュー）
2. CSS側で変化前と`.is-active`の変化後を用意しておき、JSはクラスの付け外しのタイミングだけを管理する

3. GSAPは`gsap.to()`／`from()`／`fromTo()`の**Tween**で1つの動きを作る
4. `gsap.timeline()`を使うと、複数のアニメーションの順番や時間差をまとめて管理できる
5. `onComplete`でアニメーション完了後の処理、`sessionStorage`で初回だけの処理を作れる

参考記事

- JavaScriptを使ったハンバーガーメニューを作ろう！
- GSAPを使ってアニメーションをつけよう
- GSAPの導入方法
- Tweenを扱う
- Timelineを扱う
- JavaScriptで要素を取得しよう
- JavaScriptのイベントでDOM操作をしよう
- JavaScriptでクラスを操作しよう
- 背景のスクロールを固定する方法（実務推奨）
- sessionStorage（MDN）
- GSAPの基本プロパティまとめ（ICS MEDIA）
- opacity・display・visibilityの違い
- GSAP timelineの発火タイミングの管理
- ディズニーのアニメーションの12の原則
- GSAPアニメーションギャラリー